

A Report on

IMPLEMENTATION OF CONTINUOUS INTEGRATION USING JENKINS

[BTECH-005-2023-24] ARMAN PHAUGAT

carried out as part of the course DevOps assignment

Submitted By:

Arman Phaugat

23FE10CSE00309

VI Semester

Submitted to:

Dr. Dibakar Sinha



MANIPAL UNIVERSITY
JAIPUR

**Department of Computer Science &
Engineering,
School of Computer Science and
Engineering,
Manipal University Jaipur,
*Jan-May 2026***

1. CI PRINCIPLES

1.1 Definition and Core Principles of Continuous Integration

Continuous Integration (CI) is a software development practice where developers frequently integrate their code changes into a shared repository, usually several times a day. Each integration is automatically verified by building the project and running automated tests. This helps detect errors quickly and improves software quality.

The core principles of Continuous Integration are:

- **Frequent code commits:** Developers commit code regularly instead of working in isolation for long periods.
- **Automated builds:** Every code change triggers an automated build process.
- **Automated testing:** Tests are executed automatically to verify correctness.
- **Immediate feedback:** Developers receive instant feedback if a build or test fails.
- **Single source repository:** All code is maintained in a central version control system.
- **Fast build process:** Builds should be fast so that feedback is quick.

CI plays an important role in the Software Development Life Cycle (SDLC) because it ensures early detection of bugs, improves collaboration among team members, and maintains code quality.

1.2 Difference Between CI, CD, and Continuous Deployment

Feature	Continuous Integration (CI)	Continuous Delivery (CD)	Continuous Deployment
Purpose	Integrate code frequently	Prepare code for release	Automatically release to production
Automation	Build and test automation	Build, test, and deploy to staging	Build, test, and deploy to production
Human Approval	Not required	Required before production	Not required
Risk Level	Low	Medium	High

Table 1. Difference Between CI, CD, and Continuous Deployment

Example:

- In CI Development Code Pushed → Jenkins build the code and run
- In CD Code is Pushed → Tested → Deployed to Staging Environment
- In Continuous Deployment Code is pushed → Tested → Deployed directly to server automatically

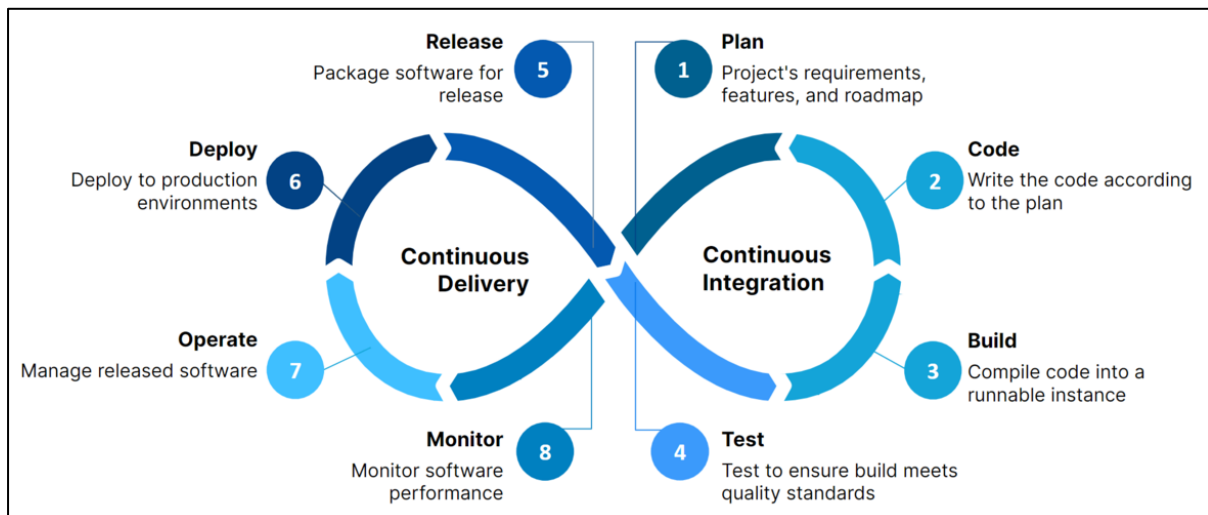


Figure 1. Working of CI/CD pipeline

1.3 Benefits and Challenges of Implementing CI

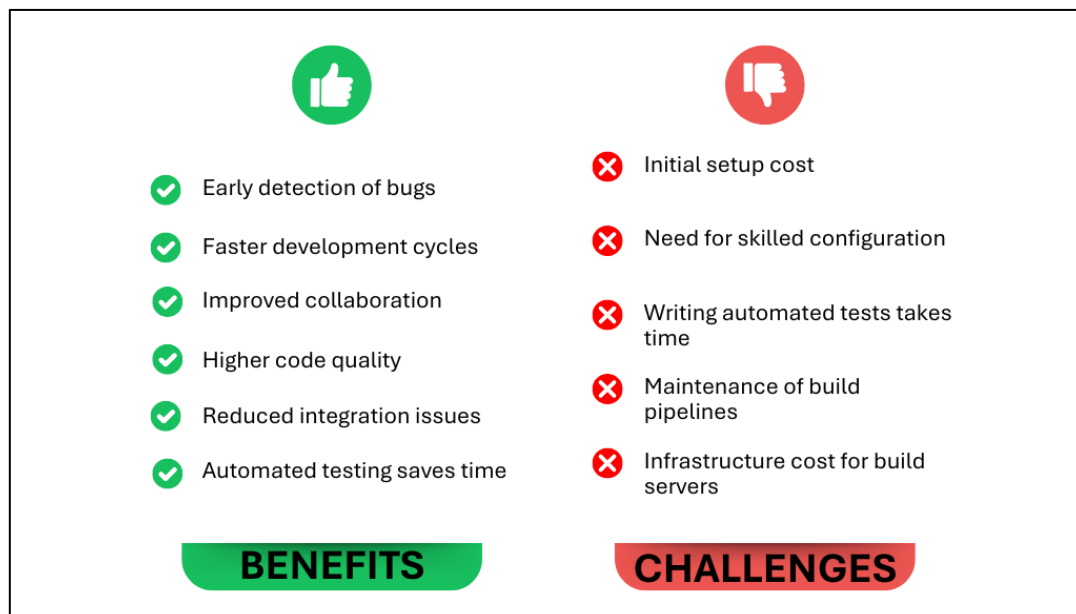


Figure 2. Analysing the Benefits and Challenges of CI

In conclusion, **Continuous Integration** offers significant advantages such as **early bug detection**, **improved collaboration**, **faster development cycles**, and **enhanced**

software quality through automation. By integrating and testing code frequently, teams can reduce integration risks and ensure stable builds. However, implementing CI **also presents challenges, including initial setup costs, the need for skilled configuration, and the effort required to create and maintain automated tests and pipelines.** Despite these challenges, the long-term benefits of reliability, productivity, and reduced error rates make Continuous Integration a valuable and essential practice in modern software development.

2. CI WORKFLOW COMPONENTS

2.1 Version Control Systems in CI

A Version Control System (VCS) such as Git allows developers to manage source code changes. In CI, VCS plays a central role because:

- All code is stored in a shared repository.
- Jenkins pulls code automatically from GitHub.
- It tracks history of changes.
- Enables collaboration and rollback of faulty code.

Popular VCS tools: Git, GitHub, GitLab, Bitbucket.

2.2 Build Automation Tools

Build automation tools compile source code and package applications automatically.

Examples:

- Maven
- Gradle
- Ant

Their significance:

- Removes manual build steps
- Ensures consistent builds
- Reduces human error
- Speeds up development

2.3 Automated Testing Frameworks

Automated testing ensures that new code does not break existing features.

Examples:

- JUnit (Java)
- PyTest (Python)
- Selenium (UI testing)

Importance:

- Improves reliability
- Finds bugs early
- Saves manual testing effort
- Enables regression testing

2.4 Artifact Repository Management

An artifact repository stores compiled binaries such as JAR, WAR, or Docker images.

Examples:

- Nexus
- Artifactory
- Docker Hub

Role:

- Stores build outputs
- Supports versioning
- Enables reuse
- Improves deployment speed

2.5 Notification Systems

Notification systems alert developers about build status.

Examples:

- Email notifications
- Slack alerts
- Microsoft Teams

Purpose: Quick feedback, Faster Bug fixing, Better Team Coordination.

3. CASE STUDY ANALYSIS

3.1 Case Study: CI Implementation in Two Organizations

Organization 1: Netflix

Netflix is one of the largest video streaming platforms in the world, serving millions of users daily. Due to its massive scale and global user base, Netflix requires highly reliable and continuously updated software systems. To achieve this, Netflix has adopted a strong Continuous Integration (CI) culture as part of its DevOps practices.

At Netflix, developers commit their code frequently to a shared Git-based repository. Every code commit automatically triggers a CI pipeline that performs a series of automated steps such as code compilation, static code analysis, and unit testing. These automated checks ensure that newly added code does not break existing functionality and meets quality standards before moving further in the pipeline.

Netflix uses cloud-based infrastructure (primarily AWS) for running its CI pipelines. This allows the company to dynamically scale its build and test environments based on workload. Instead of relying on a single build server, Netflix distributes build jobs across multiple machines, reducing build time and improving efficiency. Containerization technologies such as Docker are also used to maintain consistency across different testing environments.

Another key feature of Netflix's CI implementation is the use of microservices architecture. Each microservice has its own independent CI pipeline. This enables teams to develop, test, and deploy features independently without affecting the entire system. Automated testing includes unit tests, integration tests, and performance tests to ensure system stability.

The benefits Netflix gains from CI include:

- Faster release cycles
- Reduced system failures
- Improved developer productivity
- High software reliability
- Early detection of defects

By implementing CI at such a large scale, Netflix ensures that its platform remains stable while continuously adding new features, making CI a core pillar of its engineering culture.

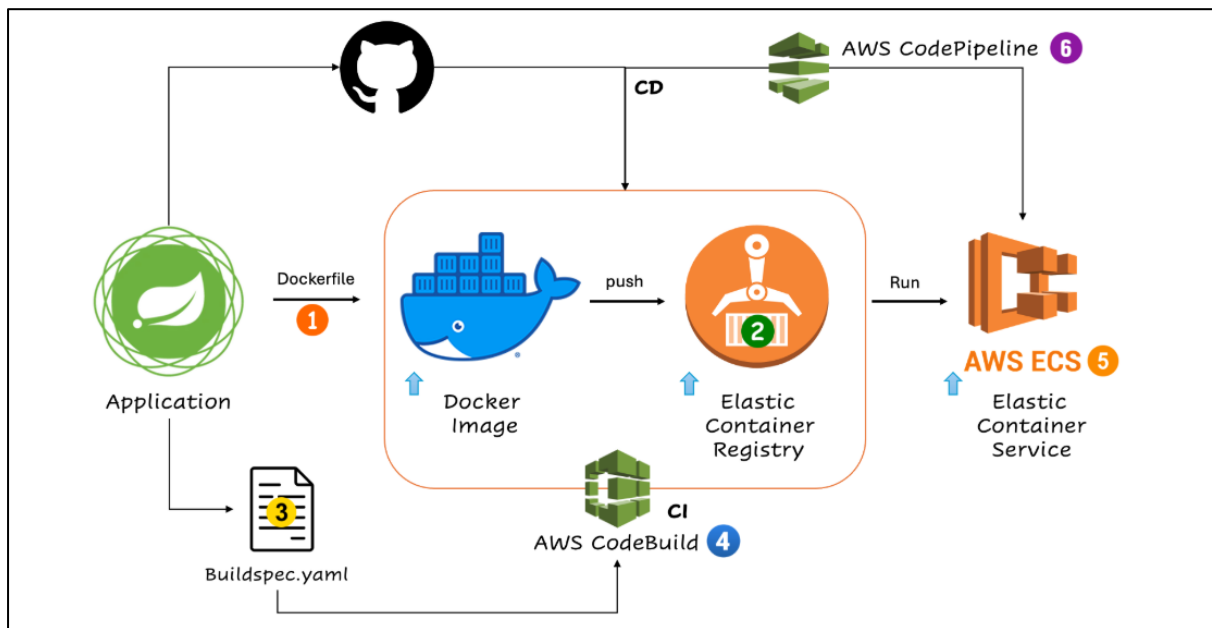


Figure 3. Netflix CI/CD Pipeline

Organization 2: Google

Google operates one of the largest software development infrastructures in the world, with thousands of engineers contributing to products such as Google Search, Gmail, YouTube, and Android. To manage this scale efficiently, Google follows a development approach based on Continuous Integration and trunk-based development.

In Google's CI model, developers commit their changes directly to a single main branch (called the trunk) instead of maintaining long-lived feature branches. Each code commit triggers automated build and test processes that verify correctness before the code is merged into the main codebase. Google uses a combination of Jenkins and internally developed CI tools to manage its build pipelines.

Google's CI system runs millions of automated tests daily, including:

- Unit tests
- Integration tests
- Regression tests
- Performance tests
- Security tests

These tests are executed in parallel across large distributed server farms. This parallel testing significantly reduces feedback time and allows developers to identify bugs quickly. Google also uses static analysis tools to detect coding errors and security vulnerabilities early in the development process.

One of the major strengths of Google's CI implementation is its automation and scalability. Builds are automatically triggered, and test results are reported instantly to

developers. If a build fails, developers receive immediate feedback so they can fix the issue before it affects other parts of the system.

The advantages Google gains from CI include:

- High code quality
- Faster feature delivery
- Reduced integration conflicts
- Improved collaboration among teams
- Better system reliability

Google's CI system enables the company to continuously improve its products while maintaining stability and performance across a very large codebase.

3.2 Comparison: Traditional vs CI-Based Development

In traditional software development models, code integration happens at the end of development cycles. Testing is mostly manual and bugs are detected late in the process, which increases the cost of fixing them. Releases are slow and risky.

In contrast, Netflix and Google use CI-based development where code is integrated multiple times a day and tested automatically. This approach reduces the chances of major integration failures and allows faster, safer releases. Automated pipelines ensure consistency, while early bug detection significantly lowers maintenance costs.

3.3 ROI (Return on Investment) Analysis for CI Implementation

Return on Investment (ROI) is used to evaluate the financial and operational benefits gained from an investment compared to its cost. In the context of Continuous Integration (CI), ROI can be measured by comparing the cost of implementing CI tools and infrastructure with the savings achieved through improved efficiency and reduced errors.

The initial costs of CI implementation include setting up CI tools such as Jenkins, configuring build servers, writing automated test scripts, and training team members. These costs may involve expenses for hardware, cloud resources, and engineering time. For example, a medium-scale organization may spend approximately ₹1,00,000 on infrastructure, tool setup, and training during the initial phase.

However, CI significantly reduces long-term operational costs. Automated testing and continuous builds help detect bugs early in the development process, which lowers the cost of fixing defects. Studies show that fixing a bug during the development phase is much cheaper than fixing it after deployment. CI also reduces manual testing effort, saving engineering time and labour costs. Additionally, faster release cycles allow

organizations to deliver features quickly, improving customer satisfaction and business value.

For instance, if an organization saves ₹3,50,000 annually due to reduced bug-fixing costs, fewer production failures, and lower manual testing effort, the ROI can be calculated as:

$$ROI = \frac{(Benefits - Investment)}{Investment} \times 100$$
$$ROI = \frac{(3,50,000 - 1,00,000)}{1,00,000} \times 100 = 250\%$$

This means that the organization gains a return of 250% on its initial CI investment. Besides financial benefits, CI also provides non-monetary returns such as improved software quality, higher developer productivity, better team collaboration, and faster time-to-market. These qualitative benefits further increase the overall value of CI adoption.

Thus, although CI requires an initial investment, the long-term savings and operational improvements make it a highly cost-effective practice for modern software development.

4. Jenkins Architecture Diagrams

4.1 Overview of Jenkins Architecture

Jenkins follows a distributed architecture that supports scalable and efficient Continuous Integration (CI) and Continuous Testing. It is primarily based on a **master-agent (slave) model**, where the main Jenkins server manages job scheduling and coordination, while multiple agent nodes perform the actual build and test execution.

This architecture enables parallel builds, better resource utilization, and separation of workload across different machines or environments.

4.2 Master-Agent (Master-Slave) Architecture

In the Jenkins architecture:

- The **Jenkins Master** is responsible for:
 - Managing users and security
 - Scheduling build jobs
 - Storing build configurations
 - Managing plugins

- Providing the web-based dashboard
- The **Jenkins Agents (Slaves)** are responsible for:
 - Executing build tasks
 - Running test cases
 - Performing deployment steps
 - Reporting build results back to the master

The master communicates with agents using secure communication protocols. Agents can be configured on physical machines, virtual machines, or containers, allowing Jenkins to scale horizontally.

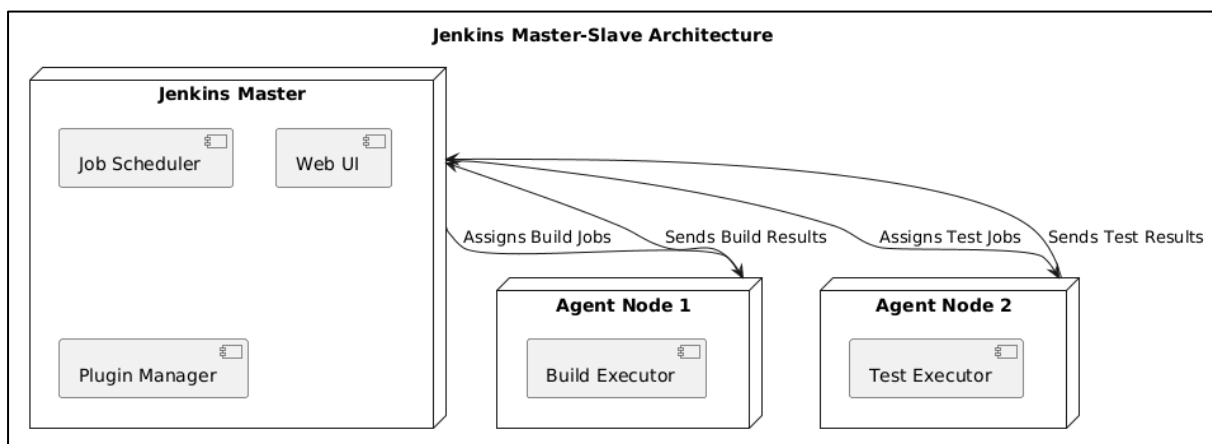


Figure 4. Jenkins Architecture Diagram showing Master and Agent nodes

4.3 Plugin Ecosystem

Jenkins has a rich plugin ecosystem that extends its core functionality. Plugins allow Jenkins to integrate with a wide range of tools used in software development and DevOps.

Examples of plugin categories shown in the architecture diagram include:

- **Source Code Management (SCM):** Git, GitHub, Subversion
- **Build Tools:** Maven, Gradle, Ant
- **Testing Tools:** JUnit, JaCoCo, Cobertura
- **Deployment Tools:** Docker, Kubernetes, SSH
- **Notification Tools:** Email Extension, Slack

Plugins act as connectors between Jenkins and external tools, enabling automation across the software development lifecycle.

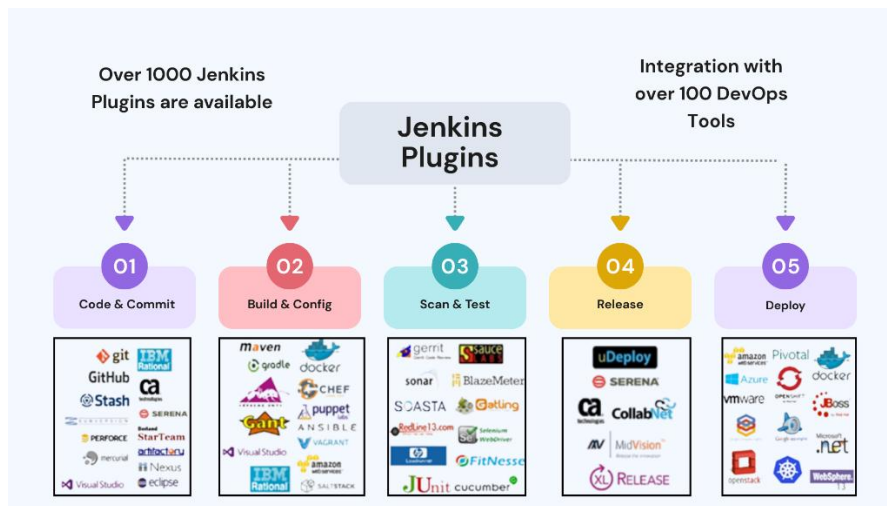


Figure 5. Jenkins Plugin Ecosystem

4.4 Integration with External Tools

Jenkins integrates with various external tools as part of the CI pipeline:

- **Version Control System (GitHub):** Developers push code to a GitHub repository.
- **Build Tool (Maven/Gradle):** Jenkins pulls the source code and compiles it.
- **Testing Framework (JUnit):** Automated tests are executed.
- **Containerization Tool (Docker):** Application is packaged into containers.
- **Deployment Platform (Kubernetes/Server):** Application is deployed to the target environment.
- **Notification Systems (Email/Slack):** Build status is communicated to developers.

This integration ensures that every code change goes through a standardized automated process.

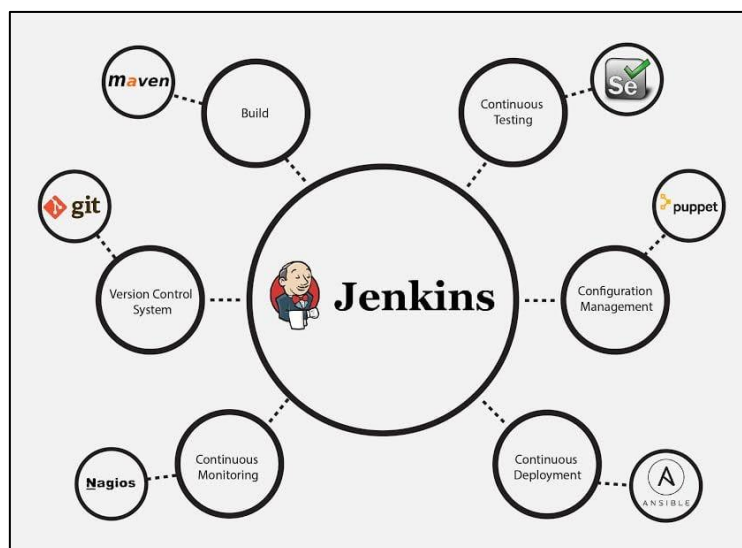


Figure 6. Jenkins Integration with other tools

4.5 Data Flow in CI Pipeline

The CI data flow illustrated in the architecture diagram follows these steps:

1. A developer commits code to the GitHub repository.
2. A webhook triggers Jenkins automatically.
3. Jenkins Master schedules the job on an available agent.
4. The agent checks out the code from the repository.
5. The build tool compiles the application.
6. Automated tests are executed.
7. Artifacts are generated and stored.
8. Deployment steps are performed (if configured).
9. Notifications are sent to the development team.

This automated pipeline ensures early detection of errors and rapid feedback.

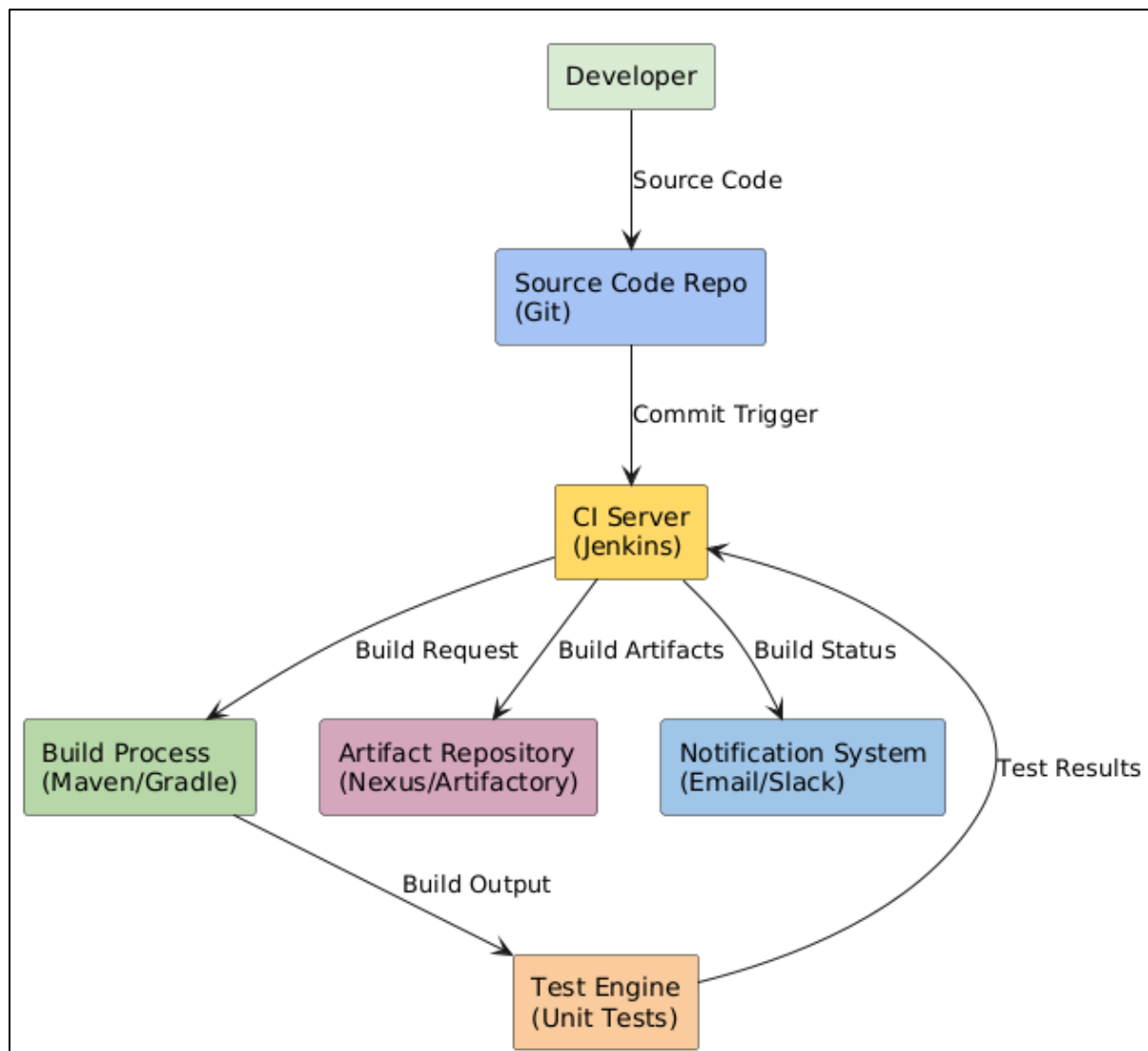


Figure 7. Data Flow in CI Pipeline

4.7 Summary

The Jenkins architecture is designed to support efficient Continuous Integration through a master-agent model, plugin-based extensibility, and seamless integration with development, testing, and deployment tools. The architecture diagram visually represents the interaction between Jenkins, external tools, and pipeline stages, demonstrating how automation is achieved across the software development lifecycle.

5. Jenkins Freestyle Job Configuration

5.1 Overview of Freestyle Project

A Freestyle project was created in Jenkins to demonstrate basic Continuous Integration functionality.

The job was configured to automatically fetch source code from a GitHub repository, execute build and test steps, and perform post-build actions such as archiving artifacts and sending email notifications. This job represents a simple CI pipeline suitable for small projects

5.2 Job Creation

The Freestyle job was created using the following steps:

1. Logged into the Jenkins dashboard.
2. Clicked on **New Item**.
3. Entered the job name and selected **Freestyle project**.
4. Clicked **OK** to create the job.

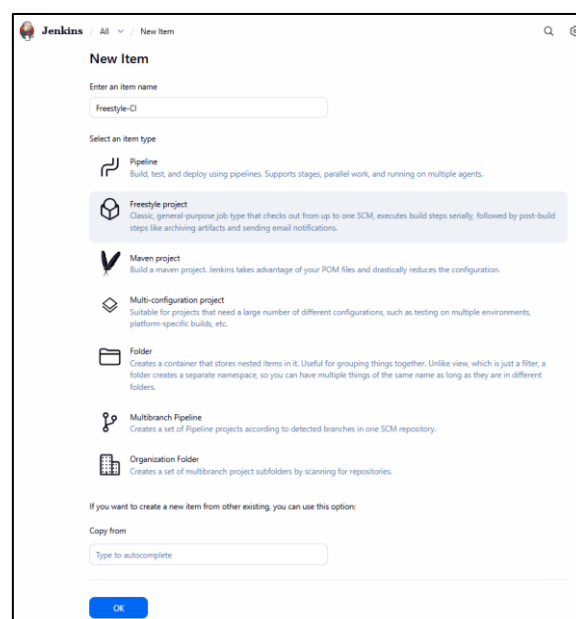


Figure 8. “Create New Item” page showing Freestyle project selected

5.3 Source Code Management (SCM) Configuration

The job was connected to a GitHub repository for automatic source code retrieval.

Configuration Details:

- Selected **Git** under Source Code Management.
- Provided the GitHub repository URL.
- Configured branch specifier (e.g., **main**).
- Added credentials if required.

This ensures that Jenkins pulls the latest code for every build.

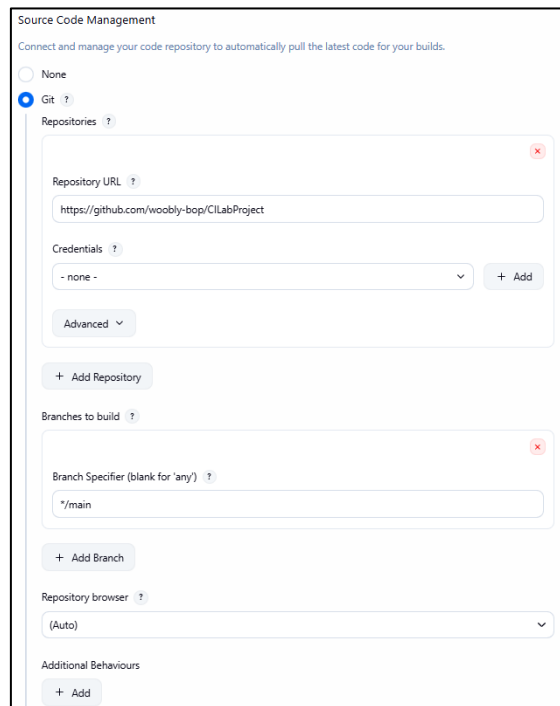


Figure 9. SCM section showing GitHub repository URL

5.4 Build Triggers Configuration

Two types of build triggers were configured:

1. Poll SCM

- Jenkins was configured to poll the repository every 5 minutes using the schedule:

```
powershell
```

```
H/5 * * * *
```

2. GitHub Webhook Trigger

- The option “**GitHub hook trigger for GITScm polling**” was enabled.

- A webhook was configured in the GitHub repository to notify Jenkins when new commits are pushed.

This allows both scheduled and event-based build triggering.

Figure 10. Build Triggers section showing Poll SCM and GitHub webhook enabled

5.5 Build Steps Configuration

The build process consisted of multiple steps:

1. Execute Windows Batch Command

A Windows batch command was added to perform basic operations such as displaying directory contents or running build scripts.

Example command:

```
powershell
set JAVA_HOME=C:\Program Files\Eclipse Adoptium\jdk-
17.0.17.10-hotspot
mvn clean package
python scripts\build.py
```

2. Run Python Script

A Python script was configured to run as part of the build process. This script performs basic operations or test logic.

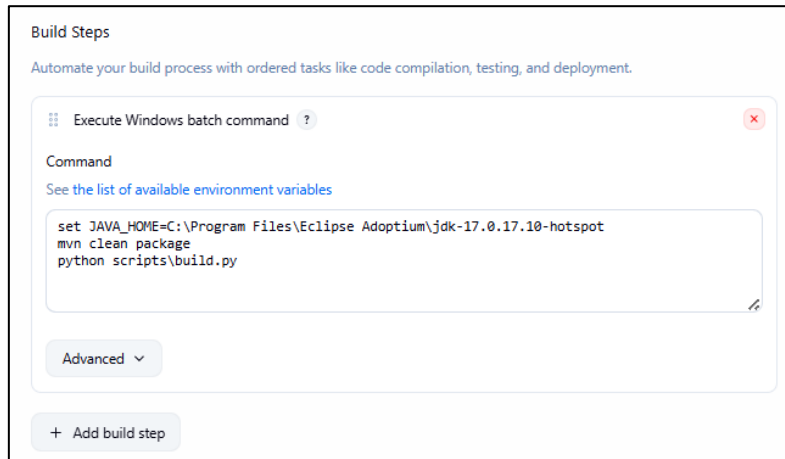


Figure 11. Build Steps section showing Windows batch command.

5.6 Post-Build Actions

After the build process, post-build actions were configured as follows:

1. Archive Artifacts

Output files generated during the build were archived for future reference.

2. Email Notifications

Email notifications were configured to inform users about build success or failure.

3. Display Test Results

Test result files were published using JUnit plugin so that Jenkins can display test reports.

These actions ensure proper reporting and traceability of build outputs.

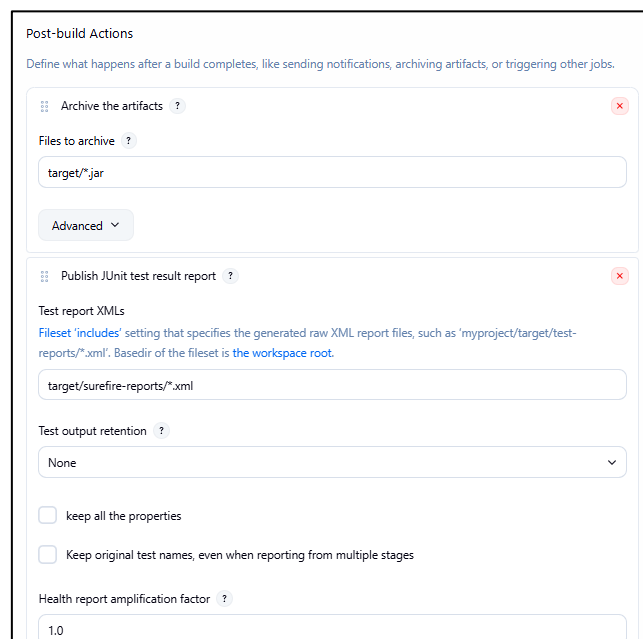


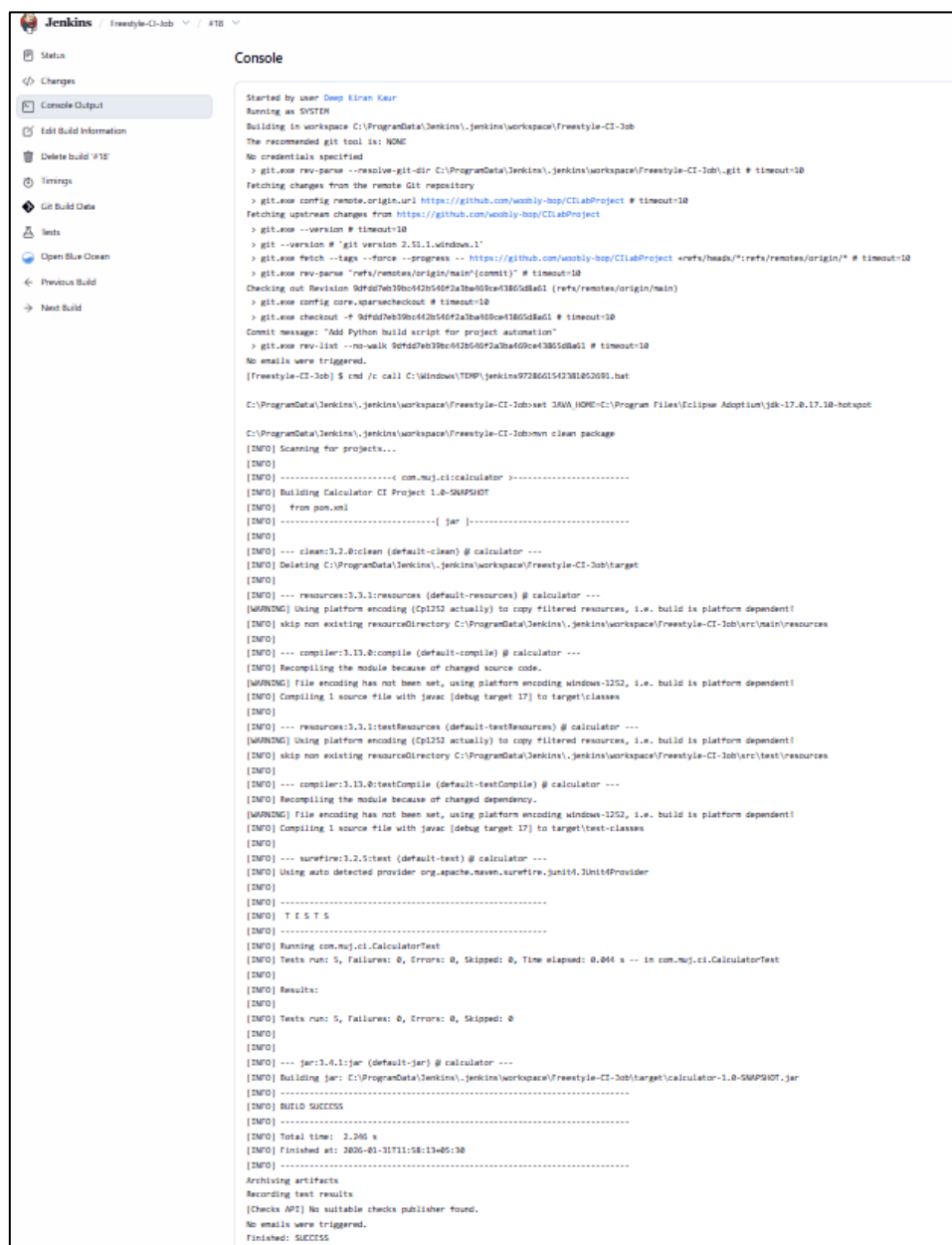
Figure 12. Post-build Actions section

5.7 Build Execution and Results

After configuration, the Freestyle job was executed manually and automatically through triggers.

Observations:

- Jenkins successfully fetched code from the GitHub repository.
- Build steps were executed without errors.
- Test results were displayed in Jenkins.
- Artifacts were archived.
- Email notification was sent upon build completion.



```
Started by user Deep Kiran Kaur
Running as SYSTEM
Building in workspace C:\ProgramData\Jenkins\jenkins\workspace\Freestyle-CI-Job
The recommended git tool is: NONE
No credentials specified
> git.exe rev-parse --resolve-git-dir C:\ProgramData\Jenkins\jenkins\workspace\Freestyle-CI-Job\.git # timeout=10
Fetching changes from the remote Git repository
> git.exe config remote.origin.url https://github.com/woohy-hop/CIJobProject # timeout=10
Fetching upstream changes from https://github.com/woohy-hop/CIJobProject
> git.exe --version # timeout=10
> git --version # 'git version 2.31.1.windows.1'
> git.exe fetch --tags --force --progress -- https://github.com/woohy-hop/CIJobProject +refs/heads/*:refs/remotes/origin/* # timeout=10
> git.exe rev-parse 'refs/remotes/origin/main:{commit}' # timeout=10
Checking out Revision 9d6dfeb39bc442b546f2a3ba609ce43805d8a1 (refs/remotes/origin/main)
> git.exe config core.sparsecheckout # timeout=10
> git.exe checkout -f 9d6dfeb39bc442b546f2a3ba609ce43805d8a1 # timeout=10
Commit message: "Add Python build script for project automation"
> git.exe rev-list --no-walk 9d6dfeb39bc442b546f2a3ba609ce43805d8a1 # timeout=10
No emails were triggered.
[Freestyle-CI-Job] $ cmd /c call C:\Windows\TEMP\jenkins9728661542381826091.bat

C:\ProgramData\Jenkins\jenkins\workspace\Freestyle-CI-Job>set JAVA_HOME=C:\Program Files\Eclipse Adoptium\jdk-17.0.17-hotspot

C:\ProgramData\Jenkins\jenkins\workspace\Freestyle-CI-Job>mvn clean package
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.muj.ci.calculator >-----
[INFO] Building Calculator CI Project 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- clean:3.2.0:clean (default-clean) @ calculator ---
[INFO] Deleting C:\ProgramData\Jenkins\jenkins\workspace\Freestyle-CI-Job\target
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ calculator ---
[WARNING] Using platform encoding (Cp1252 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory C:\ProgramData\Jenkins\jenkins\workspace\Freestyle-CI-Job\src\main\resources
[INFO]
[INFO] --- compiler:3.11.0:compile (default-compile) @ calculator ---
[INFO] Recompiling the module because of changed source code.
[WARNING] File encoding has not been set, using platform encoding windows-1252, i.e. build is platform dependent!
[INFO] Compiling 1 source file with javac [debug target 17] to target\classes
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ calculator ---
[WARNING] Using platform encoding (Cp1252 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory C:\ProgramData\Jenkins\jenkins\workspace\Freestyle-CI-Job\src\test\resources
[INFO]
[INFO] --- compiler:3.11.0:testCompile (default-testCompile) @ calculator ---
[INFO] Recompiling the module because of changed dependency.
[WARNING] File encoding has not been set, using platform encoding windows-1252, i.e. build is platform dependent!
[INFO] Compiling 1 source file with javac [debug target 17] to target\test-classes
[INFO]
[INFO] --- surefire:3.2.5:test (default-test) @ calculator ---
[INFO] Using auto detected provider org.apache.maven.surefire.junit4.JUnit4Provider
[INFO]
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO]
[INFO] Running com.muj.ci.CalculatorTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.044 s -- in com.muj.ci.CalculatorTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- jar:3.4.1:jar (default-jar) @ calculator ---
[INFO] Building jar: C:\ProgramData\Jenkins\jenkins\workspace\Freestyle-CI-Job\target\calculator-1.0-SNAPSHOT.jar
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] -----
[INFO] Total time: 2.240 s
[INFO] Finished at: 2025-05-31T11:58:13+05:30
[INFO]
[INFO] -----
Archiving artifacts
Recording test results
[Checks API] No suitable checks publisher found.
No emails were triggered.
Finished: SUCCESS
```

Figure 13. Console Output of successful build for Freestyle-Job



Figure 14. Build History showing successful build

5.8 Summary

The Freestyle job demonstrated the fundamental CI workflow using Jenkins. It integrated source code management, automated build execution, and post-build actions such as artifact archiving and notifications. This job validates the successful setup of Jenkins for Continuous Integration and automated testing.

6. Jenkins Multibranch Pipeline Configuration

6.1 Overview of Multibranch Pipeline

A Multibranch Pipeline job was created in Jenkins to automatically detect and build different branches from the connected GitHub repository. This type of job allows Jenkins to apply different build strategies for different branches, such as running full pipelines for the main branch and limited testing for feature branches.

The pipeline behaviour was defined using a **Jenkinsfile** stored within the repository.

6.2 Multibranch Pipeline Job Creation

The Multibranch Pipeline job was created using the following steps:

1. Navigated to the Jenkins dashboard.
2. Clicked on **New Item**.
3. Entered the job name and selected **Multibranch Pipeline**.
4. Clicked **OK** to create the job.

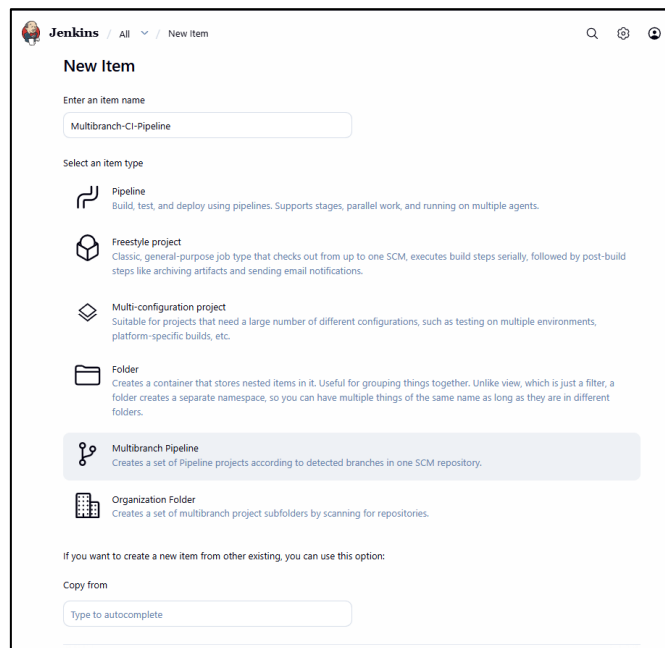


Figure 15. “Create New Item” page showing Multibranch Pipeline selected

6.3 Branch Source Configuration

The Multibranch Pipeline was connected to a GitHub repository.

Configuration Details:

- Selected **GitHub** as the branch source.
- Provided the repository URL.
- Added credentials for authentication.
- Configured the repository owner and name.

This configuration allows Jenkins to scan the repository for all existing branches.

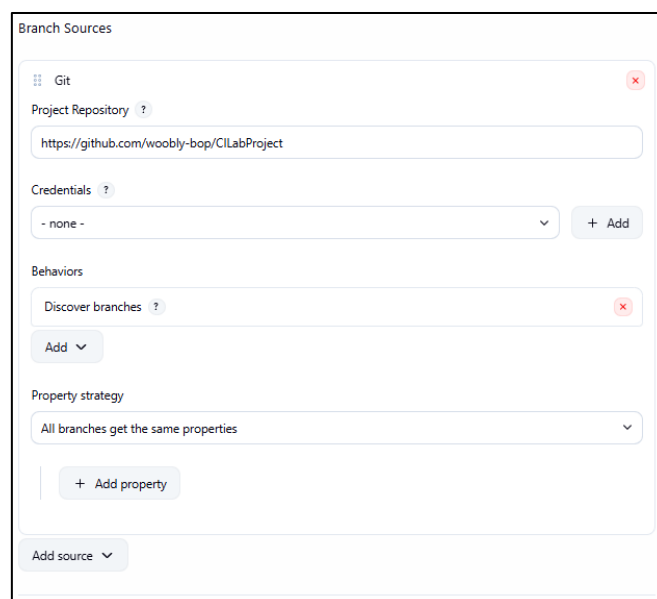


Figure 16. Branch Source configuration showing GitHub repository URL

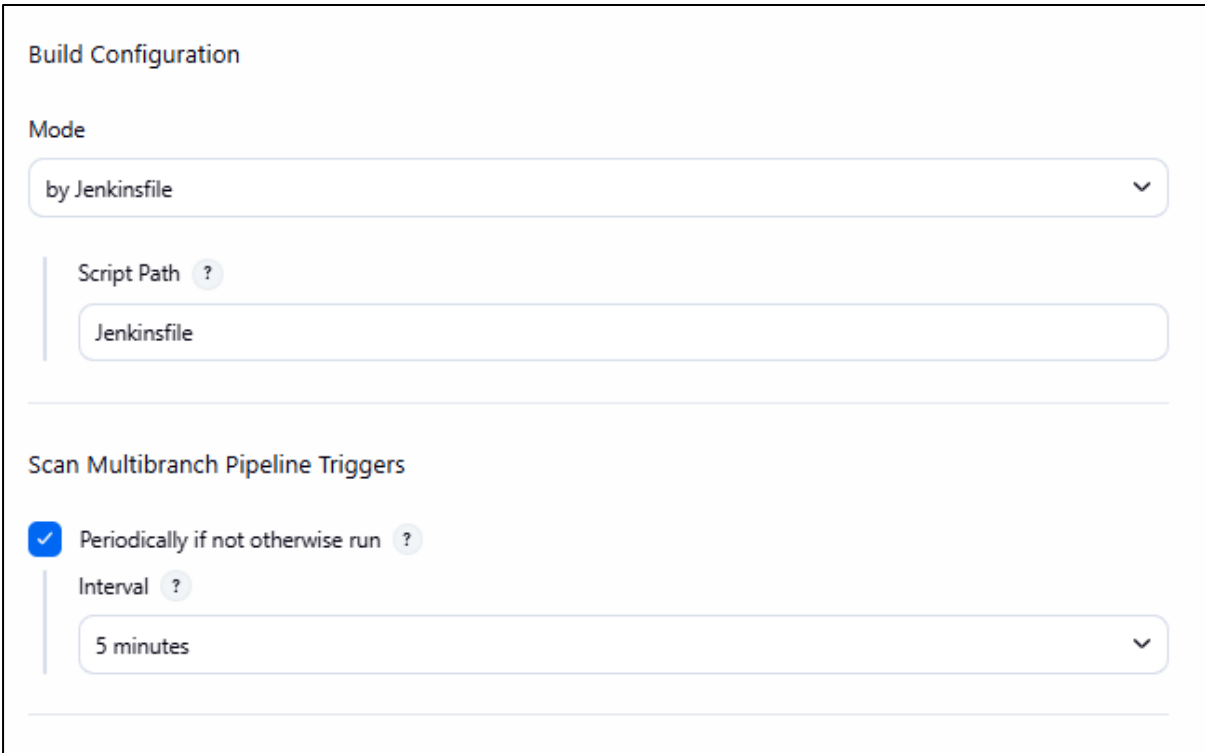
6.4 Automatic Branch Discovery

Branch discovery was enabled so that Jenkins could automatically detect new branches.

Settings:

- Enabled **Discover branches** option.
- Configured build strategies for different branch types:
 - **Main branch:** Full CI/CD pipeline
 - **Feature branches:** Test-only pipeline
 - **Release branches:** Testing and security scans

Jenkins automatically creates individual jobs for each discovered branch.



The screenshot shows the 'Build Configuration' page in Jenkins. Under the 'Mode' section, 'by Jenkinsfile' is selected in a dropdown menu. Below this, the 'Script Path' is set to 'Jenkinsfile'. In the 'Scan Multibranch Pipeline Triggers' section, the checkbox 'Periodically if not otherwise run' is checked, and the 'Interval' is set to '5 minutes' in a dropdown menu.

Figure 17. Branch Discovery settings

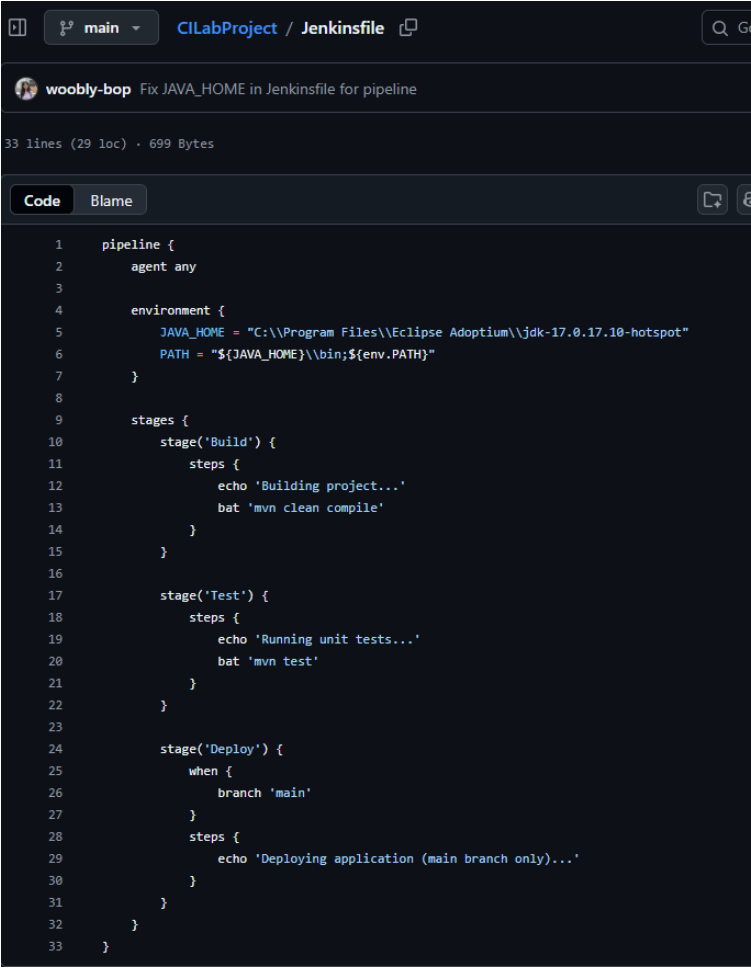
6.5 Jenkinsfile Implementation

A Jenkinsfile was created in the root of the repository to define pipeline stages.

Example pipeline stages:

- Checkout
- Build
- Test
- Security Scan
- Deploy

Conditional execution was implemented based on branch name using pipeline logic. This ensures that different branches execute different pipeline stages.



```
1 pipeline {
2   agent any
3
4   environment {
5     JAVA_HOME = "C:\\Program Files\\Eclipse Adoptium\\jdk-17.0.17-hotspot"
6     PATH = "${JAVA_HOME}\\bin;${env.PATH}"
7   }
8
9   stages {
10    stage('Build') {
11      steps {
12        echo 'Building project...'
13        bat 'mvn clean compile'
14      }
15    }
16
17    stage('Test') {
18      steps {
19        echo 'Running unit tests...'
20        bat 'mvn test'
21      }
22    }
23
24    stage('Deploy') {
25      when {
26        branch 'main'
27      }
28      steps {
29        echo 'Deploying application (main branch only)...'
30      }
31    }
32  }
33 }
```

Figure 18. Jenkinsfile from GitHub repository

6.6 Branch Indexing

Branch indexing was enabled to allow Jenkins to periodically rescan the repository.

Purpose of Branch Indexing:

- Automatically detects new branches
- Creates new pipeline jobs
- Updates deleted branches
- Triggers builds when branch structure changes

Branch indexing ensures continuous integration for all branches without manual intervention.

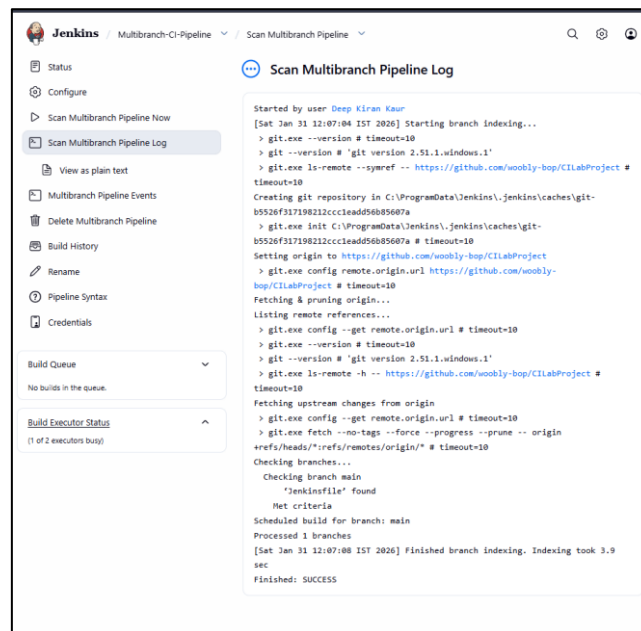


Figure 19. Branch Indexing log/output

6.7 Build Execution and Results

The Multibranch Pipeline successfully executed builds for multiple branches.

Observations:

- Main branch executed full pipeline stages
- Feature branches executed test-only pipeline
- Release branches performed testing and security checks
- Build logs and test results were visible for each branch

This demonstrates automated CI behaviour for different development workflows.

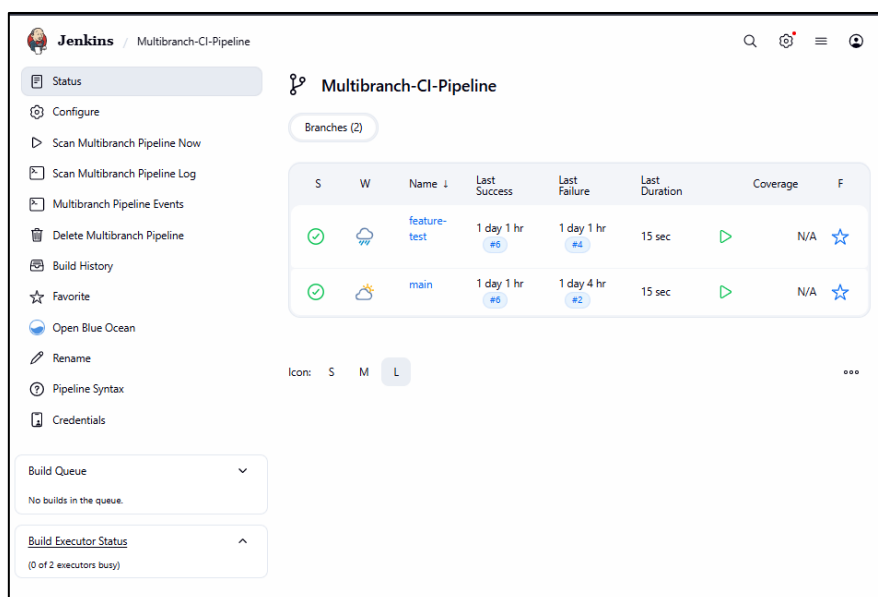


Figure 20. Multibranch Pipeline dashboard showing multiple branches

The image shows a screenshot of the Jenkins web interface. At the top, the Jenkins logo is on the left, followed by a breadcrumb trail: 'Multibranch-Cl-Pipeline' with a dropdown arrow, 'main' with a dropdown arrow, and '#3' with a dropdown arrow. Below this is a large text area containing the console output of a pipeline build. The output is a series of log messages in a monospaced font, including test results for 'com.muj.ci.CalculatorTest', a 'BUILD SUCCESS' message, and various pipeline stage markers like '[Pipeline] // stage' and '[Pipeline] { (Deploy)'. The build finished successfully at 2026-01-31T12:34:38+05:30.

```
[INFO] Running com.muj.ci.CalculatorTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.057 s -- in com.muj.ci.CalculatorTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 1.802 s
[INFO] Finished at: 2026-01-31T12:34:38+05:30
[INFO] -----
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
[Pipeline] echo
Deploying application (main branch only)...
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Figure 21. Console Output of successful pipeline build

6.8 Summary

The Multibranch Pipeline job automated the CI process for multiple branches using a single configuration.

It enabled automatic branch discovery, branch-specific build strategies, and continuous testing.

This setup reflects real-world CI/CD practices where multiple development branches are maintained simultaneously.

7. Plugin Management in Jenkins

7.1 Overview of Jenkins Plugins

Jenkins uses a plugin-based architecture to extend its core functionality.

Plugins enable Jenkins to integrate with external tools such as version control systems, build tools, testing frameworks, deployment platforms, and notification services.

In this project, essential plugins were installed and configured to support Continuous Integration and Continuous Testing workflows.

7.2 Accessing Plugin Manager

The Plugin Manager was accessed using the following steps:

1. Logged into the Jenkins dashboard.
2. Navigated to **Manage Jenkins → Manage Plugins**.
3. Used the **Available** tab to search for required plugins.
4. Installed selected plugins and restarted Jenkins when prompted.

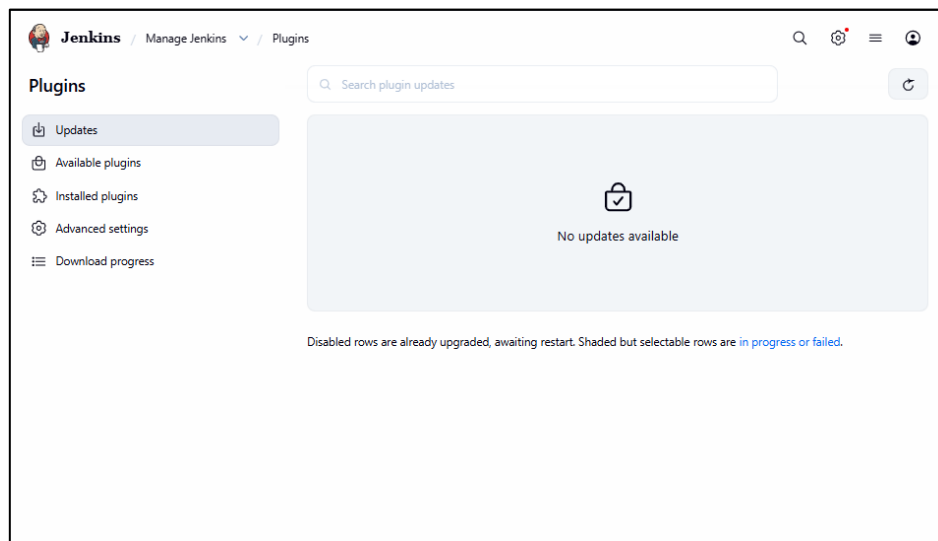


Figure 22. Jenkins Plugin Manager (Manage Plugins page)

7.3 Installed Plugins and Their Purpose

The following categories of plugins were installed as per assignment requirements:

A. Pipeline Plugins

- **Pipeline:** Used to create and manage CI/CD pipelines using Jenkinsfile
- **Blue Ocean:** Provides a modern and visual user interface for pipelines

B. Source Code Management (SCM) Plugins

- **Git Plugin:** Enables Jenkins to clone and pull code from Git repositories
- **GitHub Plugin:** Allows integration with GitHub repositories and webhooks
- **Subversion Plugin:** Supports SVN-based repositories

C. Build Tool Plugins

- **Maven Plugin:** Used for building Java-based projects
- **Gradle Plugin:** Supports Gradle build automation
- **Ant Plugin:** Used for legacy build processes

D. Testing and Code Coverage Plugins

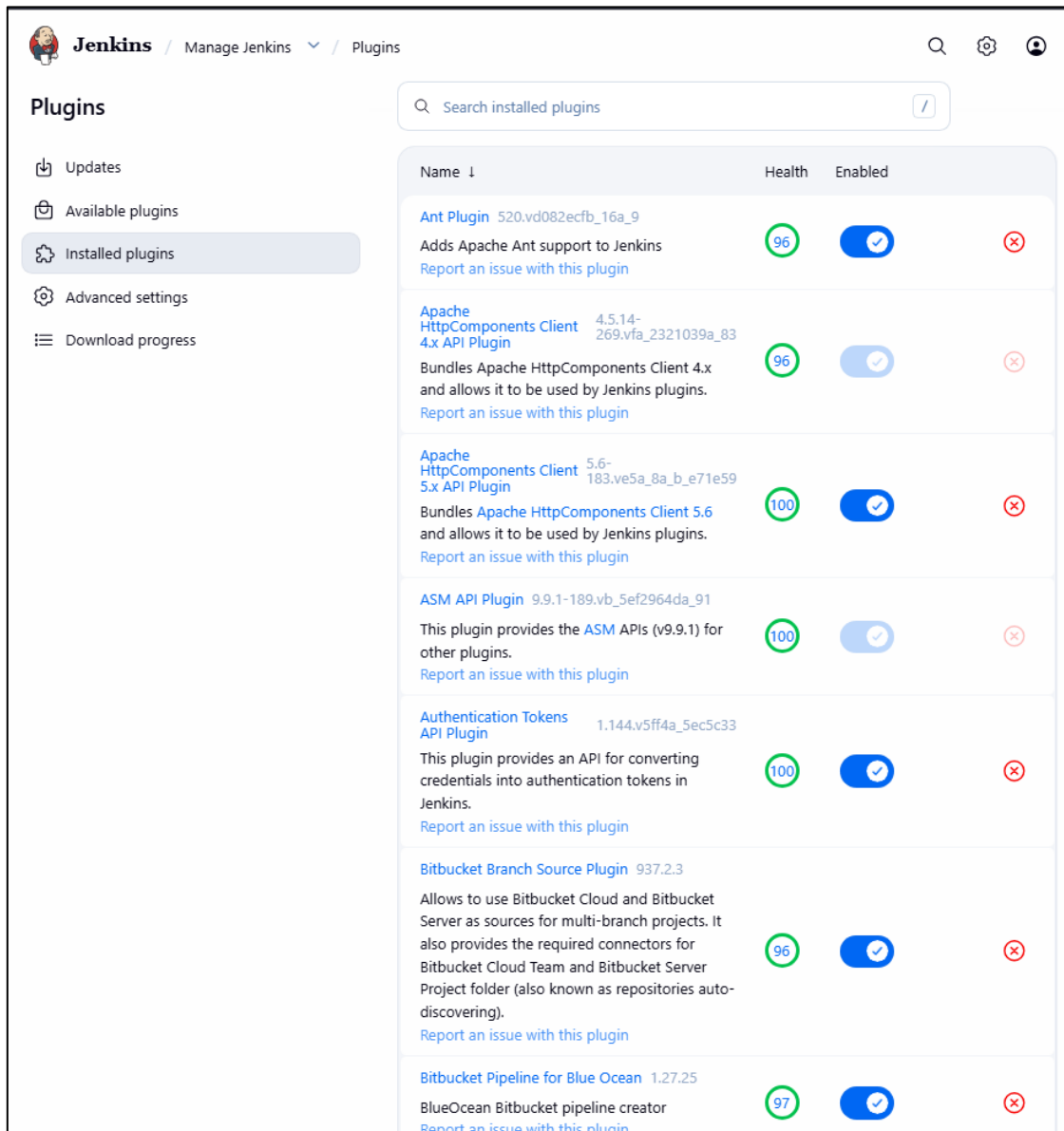
- **JUnit Plugin:** Publishes and displays unit test results
- **JaCoCo Plugin:** Measures code coverage
- **Cobertura Plugin:** Provides coverage reports

E. Notification Plugins

- **Email Extension Plugin:** Sends detailed email notifications on build status
- **Slack Notification Plugin:** Sends build alerts to Slack channels

F. Deployment Plugins

- **Docker Plugin:** Enables container-based build and deployment
- **Kubernetes Plugin:** Supports deployment to Kubernetes clusters
- **SSH Plugin:** Allows remote deployment using SSH



Name ↓	Health	Enabled
Ant Plugin 520.vd082ecfb_16a_9 Adds Apache Ant support to Jenkins Report an issue with this plugin	96	☒
Apache HttpComponents Client 4.x API Plugin 4.5.14-269.vfa_2321039a_83 Bundles Apache HttpComponents Client 4.x and allows it to be used by Jenkins plugins. Report an issue with this plugin	96	☒
Apache HttpComponents Client 5.x API Plugin 5.6-183.ve5a_8a_b_e71e59 Bundles Apache HttpComponents Client 5.6 and allows it to be used by Jenkins plugins. Report an issue with this plugin	100	☒
ASM API Plugin 9.9.1-189.vb_5ef2964da_91 This plugin provides the ASM APIs (v9.9.1) for other plugins. Report an issue with this plugin	100	☒
Authentication Tokens API Plugin 1.144.v5ff4a_5ec5c33 This plugin provides an API for converting credentials into authentication tokens in Jenkins. Report an issue with this plugin	100	☒
Bitbucket Branch Source Plugin 937.2.3 Allows to use Bitbucket Cloud and Bitbucket Server as sources for multi-branch projects. It also provides the required connectors for Bitbucket Cloud Team and Bitbucket Server Project folder (also known as repositories auto-discovering). Report an issue with this plugin	96	☒
Bitbucket Pipeline for Blue Ocean 1.27.25 BlueOcean Bitbucket pipeline creator Report an issue with this plugin	97	☒

Figure 23. Installed Plugins list showing multiple plugins

7.4 Plugin Version Details

To ensure stability and compatibility with the Jenkins LTS version, all required plugins were installed in their stable versions. The plugin versions were verified from the **Installed Plugins** tab in Jenkins Plugin Manager.

The table below lists the major plugins used in this project along with their versions:

Plugin Name	Category	Version
Pipeline	Pipeline	608.v67378e9d3db_1
Blue Ocean	Pipeline UI	1.27.25
Git	SCM	5.9.0
GitHub	SCM	1.45.0
Subversion	SCM	1303.vcfd9679fb_c12
Maven	Build Tool	3.27
Gradle	Build Tool	2.18.1203.v2c96b_1243c72
Ant	Build Tool	520.vd082ecfb_16a_9
JUnit	Testing	1396.v095840ed8491
JaCoCo	Code Coverage	3.3.7
Cobertura	Code Coverage	1.17
Email Extension	Notification	1933.v45cec755423f
Slack Notification	Notification	795.v4b_9705b_e6d47
Docker	Deployment	1308.vff6e33248305
Kubernetes	Deployment	4423.vb_59f230b_ce53
SSH	Deployment	158.ve2a_e90fb_7319

Table 2. *Plugins list with their category and versions*

These plugin versions were chosen to match the Jenkins LTS compatibility matrix and ensure reliable execution of CI pipelines.

7.5 Summary

Plugin management plays a vital role in enhancing Jenkins functionality.

By installing and configuring appropriate plugins for SCM, build tools, testing, notification, and deployment, Jenkins was transformed into a complete CI/CD automation platform. This setup supports automated builds, testing, and continuous feedback to developers.

8. Results & Observations

The Jenkins Continuous Integration environment was successfully implemented and tested as per the assignment requirements. Both Freestyle and Multibranch Pipeline jobs were executed, and the following results were observed.

8.1 Successful Installation and Configuration

- Jenkins was installed successfully on a local Windows system using the LTS version.
- Required tools such as Java JDK, Git, and Maven were properly configured.
- Security settings, including user authentication and authorization, were applied correctly.
- Email notification and webhook integration were verified.

8.2 Freestyle Job Execution Results

- The Freestyle job successfully connected to the GitHub repository and fetched the latest source code.
- Build steps, including Windows batch commands and Python script execution, ran without errors.
- Test results were generated and displayed using the JUnit plugin.
- Build artifacts were archived after successful execution.
- Email notifications were triggered based on build status.

8.3 Multibranch Pipeline Execution Results

- Jenkins automatically discovered all branches present in the GitHub repository.
- Separate pipeline jobs were created for each branch through branch indexing.
- The main branch executed the complete CI pipeline including build, test, and deployment stages.
- Feature branches executed only test stages, as configured in the Jenkinsfile.
- Release branches executed testing and security scan stages.

8.4 Plugin Functionality Observations

- SCM plugins ensured seamless integration with GitHub repositories.
- Pipeline and Blue Ocean plugins provided a clear visualization of pipeline stages.
- Testing plugins generated structured test reports and coverage results.
- Deployment plugins enabled automation of containerization and remote deployment tasks.
- Notification plugins successfully delivered build status messages.

8.5 Performance and Automation Observations

- Automated builds reduced manual intervention in the build process.
- Immediate feedback was provided after every code commit.
- Errors and test failures were detected early in the development cycle.
- Parallel execution of jobs improved build efficiency.
- The overall CI workflow was stable and repeatable.

8.6 Challenges Faced

- Initial configuration of Java and environment variables required troubleshooting.
- Plugin dependency conflicts were encountered and resolved by updating Jenkins LTS.
- Webhook triggering required correct firewall and network configuration.
- SMTP email setup required careful authentication configuration.

8.7 Overall Outcome

The Jenkins setup successfully demonstrated Continuous Integration and Continuous Testing practices.

All configured jobs executed as expected, and automation was achieved for code retrieval, build, test, and reporting processes.

The results validate the effectiveness of Jenkins as a CI tool for modern software development workflows.

9. Conclusion

This assignment successfully demonstrated the practical implementation of Continuous Integration (CI) using Jenkins. The study covered both theoretical concepts and hands-on configuration of Jenkins, highlighting its role in automating the software development lifecycle.

Jenkins was installed and configured on a local Windows system with all essential tools and plugins integrated. Freestyle and Multibranch Pipeline jobs were created to automate code retrieval, build execution, testing, and result reporting. The use of automated triggers such as webhooks and SCM polling ensured that code changes were continuously monitored and built.

The architecture diagram illustrated the master-agent model, plugin ecosystem, and integration of Jenkins with external tools such as GitHub, Maven, and testing frameworks. Plugin management enabled seamless extensibility, allowing Jenkins to support diverse CI/CD workflows.

The results confirmed that automated builds improved development efficiency, reduced manual intervention, and provided immediate feedback on code quality. Challenges related to configuration and plugin compatibility were resolved through systematic troubleshooting.

Overall, this project validated Jenkins as a powerful and flexible tool for Continuous Integration and Continuous Testing. The implemented CI environment reflects real-world DevOps practices and provides a strong foundation for further extension into full Continuous Deployment pipelines.